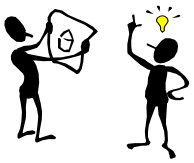
JAVA 101: CLASSES AND OBJECTS



Contents

1 Fundamental OOP Concepts (abstraction, encapsulation, inheritance, polymorphism) 2
2 Anatomy of a Java class declaration 3
2.1 Class declaration 3
2.2 Simple program class 3
2.3 Method library class 4
3 OBJECT-ORIENTED PROGRAMMING WITH CLASSES AND OBJECTS 6
4 Creating a class for objects step by step 9
Version 1: Class for objects, no constructor, no encapsulation 9
Version 2: Class for objects, constructor and toString method added, no encapsulation 10
Version 3: Class for objects, properly encapsulated final version 11
5 Reference types, object references, and the reference copy 12
6 Arrays of object references 13
7 Lists of object references (using the ArrayList class) 14
8 Wrapper classes 15
9 Classes that have both instance-level and class-level features 16
10 Introduction to *interfaces* and *lambda expressions*: sorting arrays and lists 17
10.1 Sorting arrays of primitive types 17
10.2 Sorting an array with the compareTo method (using the Comparable interface) 18
10.3 Sorting an array with a Comparator object (using the Comparator interface) 19
10.4 Sorting an array with a lambda expression 19
10.5 Sorting Lists 21
10.6 A short example of a Java interface 21
10.7 An example of a sort algorithm 22
11 Generic classes: ArrayList under the hood (for advanced learners) 23
List of key concepts. List of Java keywords and Java API classes used in the examples 24

Diagrammatic presentation of the classes for objects declared in the short examples

Stopwatch example

Stopwatch
- double startTime - double elapsedTime - boolean isRunning
+ Stopwatch() + void start() + void stop() + double getElapsedTime()

UML class diagram notation explained

Class name

Fields

Constructor

Methods

Visibility modifiers:
'-' means private
'+' means public

Names of static fields and static methods are underlined.

Course example

Course
- String code - String name
+ Course(String code, String name) + String getCode() + String getName() + void setCode(String code) + void setName(String name) + String toString()

Error message example

ErrorMessage
- String message - int <u>messageCount</u>
+ ErrorMessage(String messageText) + int <u>getMessageCount</u>() + String toString()

1 Fundamental OOP Concepts (abstraction, encapsulation, inheritance, polymorphism)

In the object-oriented programming paradigm (OOP)[1], the ***object*** is the fundamental element of abstraction and the core building block for programs. The aim is to organize the program into smaller parts (objects) that are as easy as possible to understand, test, maintain, and reuse.

Abstraction is a technique for reducing complexity of computer systems to a manageable level.

In *class-based* OOP languages (Java, C#, ...), we design and define ***classes*** for creating objects. That is, in Java, every object is created from a class.

1. In the Java run-time environment, there is a predefined set of classes like *Scanner, DecimalFormat, String* etc. and you can create any number of new classes. The content of the predefined Java class library is documented in the **Java API Specification**.
2. Related Java classes can be bundled into ***packages*** to make the classes easier to find and use[2], to avoid class name conflicts, and to control access to individual classes.
3. An executable Java program or any set of *compiled Java classes*[3] and related files (images etc.) can be packaged in a ***Java Archive File*** (JAR file) for distribution of the software[4].

Encapsulation is a mechanism of *wrapping the data and code acting on the data together as a single unit called object.* That is, an object can encapsulate both data (several different data items in a single object) and behaviours. The behaviours of an object are defined by defining the methods of the object. Encapsulation supports robustness, testability, maintainability, and reusability of the code.

1. To promote encapsulation, **the data held in an object should be declared with *private visibility***. That is, the data held in an object should be manipulated only by that object.
2. The set of methods of an object that have *public visibility* define the interface between the object and the rest of the system.

Visibility	***Instance fields***	***Instance methods***
`private`	Enforce encapsulation	Provide services to the other methods of the object
`public`	***Violate encapsulation***	Provide services to the rest of the system

Inheritance is about *reusability* and *programming by difference*. In Java, you can derive any number of new classes from an existing class[5]. In doing this, you can reuse the fields and methods of the existing class without having to design, write, test, and debug them again. The class, which inherits the properties of other, is known as *subclass* (derived class / child class) and the class whose properties are inherited is known as *superclass* (base class / parent class).

Polymorphism[6] is a quite advanced theoretical concept. It is something that we can achieve by using the concrete OOP features of the programming language. Polymorphism is mainly about enabling different types of objects to share the same service interface (or a part of it) and respond to the same service request in their own specific way. In Java, polymorphism can be

1. **Subtype polymorphism** (achieved by applying *inheritance* and *method overriding*, or implementing *interfaces*)
2. **Parametric polymorphism** (achieved by using *generics*)
3. **Ad hoc polymorphism** (achieved by applying *method overloading*).

---

[1] See https://en.wikipedia.org/wiki/Programming_paradigm for more information about programming paradigms.
[2] For example, '***import java.util.Scanner***' allows the use of the class Scanner from the ***java.util*** package.
[3] The Java source texts are ***not*** included in the JAR file for distribution.
[4] Examples of this type of software distribution are JDBC drivers for SQL Server, MariaDB and Sqlite. They are distributed as single JAR files that you include in your Java projects.
[5] If we do not explicitly derive a class from another class, then Java derives the class from a class called *Object*.
[6] See https://en.wikipedia.org/wiki/Polymorphism_(computer_science) for more details. To discuss the details of the concept of polymorphism is not in the scope of this presentation.

2 Anatomy of a Java class declaration

2.1 Class declaration

All Java programs consist of one or more *classes*[7]. There is no Java code outside of class declarations[8].

A Java class declaration requires the following items:

1. Class modifier(s) - Typically, the `public` visibility modifier
2. The `class` keyword
3. Class name - Starts with a capital letter by the Java naming convention
4. Class body enclosed in braces.

That is, the smallest possible class declaration is the following:

```
class MyClass {
}
```

The very basic (*optional*) items that we write inside of a class body are the following:

Class-level items

- Fields - **class fields** (`static`), also called *class variables*
- Methods - **class methods** (`static`)

Instance-level items

- Fields - **instance fields** (*non-static*[9]), also called *instance variables*
- Methods - **instance methods** (*non-static*)
- Constructors

2.2 Simple program class

The code of a very ***simple standalone Java program*** can be written in a single class declaration.

SimpleProgram.java (A simple program class)

```
public class SimpleProgram {
    public static void main(String[] args) {
        System.out.println("This is a minimal complete Java program!");
    }
}
```

The above program class ***SimpleProgram*** is as simple as it can be.

The access (visibility) modifier of the *SimpleProgram* class is `public`**.** That is, we do not want to limit the visibility of this class[10].

The access modifier of the *main* method is public. That is, we do not want to limit the visibility of this method. The `static` modifier of the main method says that the method is a ***class method***. A class method can be invoked *without creating an object from the class*.

---

[7] **NB!** The term 'class' is ambiguous in software development. In this guide, the term '**class**' normally refers to ***a Java class declaration***.

[8] Normally, the source code of each class is saved in a separate text file. The name of the text file must be the same as the class name. The file name extension must be ***.java***. We consider Java *interfaces* to be special classes, too.

[9] *Non-static* = defined without the `static` keyword. By default, fields and methods are non-static.

[10] If we leave out the `public` keyword, then this class is not accessible from outside of the package that it belongs.

2.3 Method library class

A ***method library class*** is a class used to store a group of related reusable, static methods. A typical method library class does *not* have the *main* method.

Below is a part of the declaration of the *Math* class from the Java class library[11]. In the class declaration below, there is one ***class field*** (*PI*) and one ***class method*** (*max*).

Math.java (A method library class)

```
package java.lang;

public class Math {
    public static final double PI = 3.14159265358979323846;

    public static double max(double a, double b) {
        return (a > b) ? a : b;
    }
}
```

The `public` visibility modifier of the class Math is `public` says that anyone can use this class[12].

The modifiers of the field ***PI*** are `public`, `static`, and `final`.

*A **class field** is a single variable whose existence is not dependent on any object.*

- The `public` modifier says that anyone can use this field[13].
- The `static` modifier says that this field is a ***class field***.
- The `final` modifier says that the value of this field cannot change after initialization. That is, ***PI*** is defined to be a ***constant***. By the Java naming convention, names of constants are written in all uppercase. A constant can be `public` because its value cannot be changed[14].

The modifiers of the method ***max*** are `public` and `static`.

- The `public` modifier says that anyone can call this method.
- The `static` modifier says that this method is a ***class method***.

To access a public class field/method from outside of the class declaration you must write the name of the class and a dot before the field/method name. You can access the public fields and call the public methods of a method library class from any program. That is, you *design, write and test the code only once and then you can use the code many times – without duplicating the code.*

DemoProgram.java (A simple program class)

```
public class DemoProgram {
    public static void main(String[] args) {
        int radius = 10;

        System.out.println("Area of the circle is " + Math.PI * radius * radius);
        System.out.println(Math.max(3.14, Math.PI));
    }
}
```

[11] The Math class belongs to the ***java.lang*** package. For convenience, the Java compiler *automatically imports* the *java.lang* package because the package contains classes that are fundamental to the design of the Java language.

[12] The available visibility modifiers for classes are the following:

`public`	(the class is visible to anyone)
no visibility modifier	(the class is visible only in the code of the classes of the same package)

[13] The available visibility modifiers for fields and methods are the following:

`public`	(visible to anyone)
`protected`	(visible in the code of the classes of the same package and in the code of all subclasses
no visibility modifier	(visible in the code of the classes of the same package)
`private`	(visible in the code of the class)

[14] Another example of a public constant is the ***length*** field of an array. It gets its final value when the array is created.

Here is another example of a method library class.

Keyboard.java (A simple method library class)

```
import java.util.Scanner;

public class Keyboard {
    private static Scanner input = new Scanner(System.in);

    public static double readDouble(String prompt) {
        System.out.print(prompt);
        return Double.parseDouble(input.nextLine().replace(',', '.'));
    }

    public static int readInt(String prompt) {
        System.out.print(prompt);
        return Integer.parseInt(input.nextLine());
    }

    public static String readString(String prompt) {
        System.out.print(prompt);
        return input.nextLine();
    }
}
```

The modifiers of the field ***input*** are **private** and **static**.

- The **private** modifier says the field is visible in the code of this class only. The field is for internal use in the code of this class.
- The **static** modifier says that this field is a ***class field***.

All the methods of the *Keyboard* class are public class methods. That is, we don't have to create an object of this class to use these methods.

PriceProgram.java (A simple program class)

```
public class PriceProgram {
    public static void main(String[] args) {
        String productName = Keyboard.readString("Enter product name: ");
        double price = Keyboard.readDouble("Enter price: ");
        int quantity = Keyboard.readInt("Enter quantity: ");

        System.out.println("The total price of " + quantity + " " +
            productName + "(s) with VAT is " + (1.24 * price * quantity));
    }
}
```

NB! Java is not a 100 % object-oriented language[15]. This far, our examples have been primarily demonstrating *procedural programming* in Java.

- In *procedural programming* the focus is on breaking down a programming task into a set of variables, data structures (e.g. arrays), subroutines (e.g. methods) and modules (e.g. method libraries).
- In *object-oriented programming* (OOP) the focus is on breaking down a programming task into ***objects*** that encapsulate both the data and the methods. That is, in OOP we define and create objects of our own types.

[15] In a *pure* class-based OOP-language, (1) all classes are *classes for objects* and (2) *all data is presented as objects*. In Java (and C#), in addition to classes for objects, we can create *simple program classes* and *method library classes* that are not used as classes for objects. In addition to objects, Java manages data that is presented by using primitive types (e.g. int, double, boolean, and char). In addition to imperative programming and object-oriented programming, Java supports event-driven programming and functional programming (to some extent only).

3 OBJECT-ORIENTED PROGRAMMING WITH CLASSES AND OBJECTS

In *object-oriented Java programming*, we create ***classes for objects*** and ***instantiate***[16] these classes. The aim is to organize the program into such classes/objects that are as easy as possible to understand, test, maintain, and reuse.

An object has ***behaviors*** and ***attributes***. The values of the object's attributes define its ***state***. Only the object itself should be allowed to directly change its state.

A software object can be used to represent a real object in the problem domain. For example, we can create one object per bank account, each with ***behaviours*** (deposit money, withdraw money, get balance etc.) and ***attributes*** (account number, balance etc.) that we need to represent. The values of the object's attributes define its ***state***. For example, a part of a bank account's state is its current balance.

In Java, we create objects by instantiating *classes for objects*.

- The declaration of a class for objects acts as a ***blueprint of an object***. It specifies what kind of behaviours and attributes each object created from the class will have.
- In Java, the *public* ***methods*** of an object define its behaviours.
- In Java, the ***fields*** of an object define its attributes.
- We can create any number of objects from one class.

In the code of a typical Java *class for objects*, there are the following instance-level items:

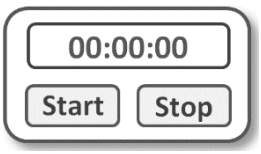
- Fields — instance fields (*non-static*), also called *instance variables*
- Methods — instance methods (*non-static*)
- Constructors — also called *object constructors*.

Stopwatch example

Let's consider a concrete example that demonstrates and clarifies the idea of software objects and the key concepts mentioned this far.

Suppose that we want to measure elapsed time in a Java program. In the real world, we can use a stopwatch for precise timing. A very simple stopwatch has a digital display that can be started and stopped at will for exact timing. Let's apply the object-oriented approach and determine the behaviours and attributes that we need to represent in a Stopwatch object.

A real-world stopwatch → Java class for **Stopwatch** objects



Attributes	*Fields (private)*
* elapsed time	`elapsedTime`
Behaviors	*Methods (public)*
* start	`start()`
* stop	`stop()`
* get elapsed time	`getElapsedTime()`

Next, we have to consider how we can do the elapsed time calculation in Java. The class method `currentTimeMillis` of the `System` class returns *the current time in milliseconds*. If we use the `currentTimeMillis` method to get the start time and end time, then we can calculate the *elapsed time in seconds* as follows: *(end time - start time) / 1000.* If we save the start time and elapsed time to variables of type *double*, then we do not lose the fractions of a second in our calculations. To keep the example short and simple, we do not consider converting the result to hours, minutes, and seconds.

[16] To ***instantiate*** = to represent (an abstraction) by a concrete instance
To Instantiate a class = to create an instance of the class = to create an ***object*** from the class

Here is the source code of our simplified *Stopwatch* class.

Stopwatch.java (A class for objects)

```
public class Stopwatch {
    // 1. Instance fields (must be non-static)
    private boolean isRunning;
    private double startTime;
    private double elapsedTime;

    // 2. Constructor (Java invokes the constructor automatically on a newly created object)
    public Stopwatch() {
        isRunning = false;
        startTime = 0;
        elapsedTime = 0;
    }

    // 3. Instance methods (must be non-static)
    public double getElapsedTime() {
        return elapsedTime;
    }

    public void start() {
        isRunning = true;
        elapsedTime = 0;
        startTime = System.currentTimeMillis();
    }

    public void stop() {
        if (isRunning) {
            elapsedTime = (System.currentTimeMillis() - startTime) / 1000;
            isRunning = false;
        }
    }
}
```

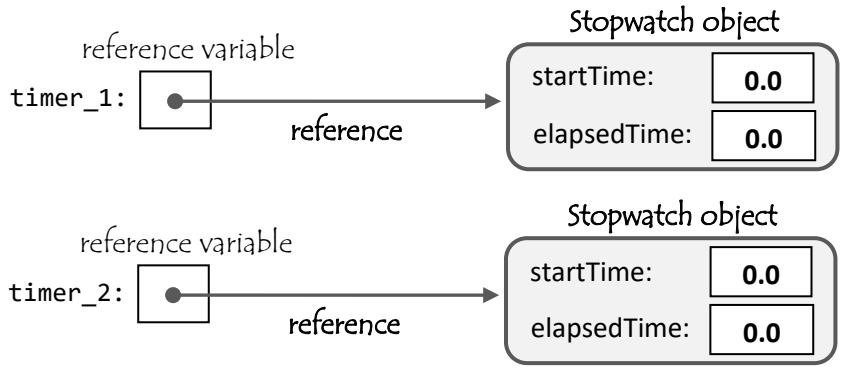
The ***new*** operator always creates exactly one object.

To create a new object from a class, we use the ***new*** operator, which returns a *reference to the new object*. We can create any number of Stopwatch objects.

```
Stopwatch timer_1 = new Stopwatch();
Stopwatch timer_2 = new Stopwatch();
```

NB! An object itself is ***never*** stored in a variable. Instead, a variable can store a ***reference to an object***. If a variable can hold a reference to an object, we can call the variable ***reference variable***[17].

A ***reference*** is a 'link' (memory address) that indicates where an object is stored.



A ***constructor*** is a special method that initializes a newly created object

The system automatically invokes an *object* ***constructor*** when an object is being created from the class. An *object constructor is* a special type of method that has the ***same name as the class*** and ***no return type***. The constructor of the class should initialize the fields of the newly created object. If we do not explicitly define a constructor for our class, then Java creates behind the scenes a *default constructor* that takes no parameters and performs no special operations.

[17] A reference variable is like an entry in an address book. The entry tells you where to find what you are looking for.

In the first example, we use the ***Stopwatch*** class and measure how long it takes to iterate a loop one billion times.

MeasureElapsedTime.java (A simple program class)

```
public class MeasureElapsedTime {

    public static void main(String[] args) {
        Stopwatch timer = new Stopwatch();

        timer.start();
        for (int i = 1; i <= 1_000_000_000; i++) {
        }
        timer.stop();

        System.out.println("Elapsed time: " + timer.getElapsedTime() + " s");
    }
}
```

In the next example, we measure how long it takes to join 100000 short strings with the '+' operator and how long it takes to join 100000 short strings with a StringBuilder. For this purpose, we create two separate objects from the ***Stopwatch*** class.

StringBenchmark.java (A simple program class)

```
public class StringBenchmark {

    public static void main(String[] args) {
        Stopwatch timer_1 = new Stopwatch();
        Stopwatch timer_2 = new Stopwatch();
        String firstText = "";
        StringBuilder secondText = new StringBuilder("");

        timer_1.start();
        for (int i = 1; i <= 100_000; i++) {
            firstText = firstText + "Hello! ";
        }
        timer_1.stop();

        timer_2.start();
        for (int i = 1; i <= 100_000; i++) {
            secondText.append("Hello! ");
        }
        timer_2.stop();

        System.out.println("Using the '+' operator: " +
            timer_1.getElapsedTime() + " seconds");

        System.out.println("Using a StringBuilder: " +
            timer_2.getElapsedTime() + " seconds");
    }
}
```

NB! Normally, we distribute Java classes in their machine-readable bytecode (***.class*** files). That is, ***the source code*** of the ***Stopwatch*** class (Stopwatch***.java***) ***should exist only once*** (it should not be duplicated). After we have compiled and thoroughly tested the code, we can distribute the class by copying the file Stopwatch***.class*** to other Java projects. If we have to make any changes to the code later, then we should *modify the original source code*, test the new version thoroughly and then distribute the new version of the file Stopwatch***.class***.

4 Creating a class for objects step by step

Version 1: Class for objects, no constructor, no encapsulation

Let's develop a simple class for objects step by step. The declaration of the ***Course*** class is the blueprint of a *Course* object. Each Course object should contain *course code* and *course name*.

Course.java (*First version of our class for objects*)

```
public class Course {
    // Instance fields (must be non-static)
    public String code;
    public String name;
}
```

Any field declared without the `static` modifier is an *instance field*. Instance fields are associated with instances of the class (objects). Thus, every Course object we create has its own instance fields called ***code*** and ***name***. To keep the first version of the class as simple as it can be, we do not include any methods or constructors[18] yet and we define the fields with `public` visibility[19].

We can *instantiate* the class (create an object from the class) with the *new* operator and access the object's public fields as follows:

```
Course firstCourse = new Course();
Course currentCourse = new Course();

firstCourse.code = "swd1tf001";
firstCourse.name = "Introduction to software engineering";

currentCourse.code = "swd4tf014";
currentCourse.name = "Programming 1";

System.out.println(firstCourse.code + ": " + firstCourse.name);
System.out.println(currentCourse.code + ": " + currentCourse.name);
```

The code above prints the following:

```
swd1tf001: Introduction to software engineering
swd4tf014: Programming 1
```

[18] (1) Since we do not explicitly define a constructor for our *Course* class, the Java compiler will create behind the scenes a default constructor that takes no parameters and performs no special operations (in addition to implicitly calling its direct superclass's parameterless constructor). (2) The *Course* class's direct superclass is the *Object* class since we do not explicitly derive the *Course* class from any other class with the `extends` keyword. This keyword, superclasses/subclasses, and inheritance will be discussed later in detail (in another guide).

[19] Normally, all instance fields should be `private`. In this example, we will later change them to `private`.

Version 2: Class for objects, constructor and toString method added, no encapsulation

Let's improve the Course class as follows:

- Simplify the initialization of the object's instance fields by defining a ***constructor*** that accepts two input parameters and copies their values to the instance fields of the object.
- Simplify the printing of the object's data (*for testing purposes only*) by defining the ***toString method*** that returns a representation of the object's data as a String.

Course.java (*Second version of our class for objects*)

```
public class Course {
    // 1. Instance fields (must be non-static)
    public String code;
    public String name;

    // 2. Constructor (Java invokes the constructor automatically on a newly created object)
    public Course(String courseCode, String courseName) {
        code = courseCode;
        name = courseName;
    }

    // 3. Instance method (must be non-static)
    public String toString() {
        return code + ": " + name;
    }
}
```

In Java, the purpose of the ***toString*** method is to return a textual representation of the object.

Any method declared without the `static` keyword is an ***instance method***. An instance method operates always on an instance of a class (an object). That is, you cannot call an instance method without a reference to an object[20].

Now we can instantiate the *Course* class with parameters and print course data as below.

```
Course firstCourse =
    new Course("swd1tf001", "Introduction to software engineering");
Course currentCourse = new Course("swd4tf014", "Programming 1");

System.out.println(firstCourse);
System.out.println(currentCourse);
```

The code above prints the following[21]:

```
swd1tf001: Introduction to software engineering
swd4tf014: Programming 1
```

[20] For example, *Scanner* is a class for objects. If you try to execute the call `Scanner.nextline()`, you get the following error message: "Cannot make a static reference to the non-static method nextLine() from the type Scanner".

[21] When we want to get a *Course* object's data to be printed, we can simply pass a reference to the *Course* object to the *println* method as follows:

```
System.out.println(firstCourse);
```

One of the overloaded versions of the *println* method accepts a reference to an object as its input parameter. The method automatically calls the ***toString*** method of the object to get a String to be printed. Of course, we can alternatively call the *toString* method explicitly as below.

```
System.out.println(firstCourse.toString());
```

Version 3: Class for objects, properly encapsulated final version

Finally, let's enforce ***encapsulation***[22] of the *Course* class as follows:

- Define all the fields with **`private`** visibility instead of **`public`** visibility. A field that is defined with **`private`** visibility can be accessed only by the methods of the class.
- Provide **`public`** ***getters*** (*accessor methods*) and ***setters*** (*mutator methods*) for the fields. By the Java naming convention, the getter names have the form ***`get`***X and setter names have the form ***`set`***X.

'***this***' is a reference to the current object.

Course.java (*Final version of our class for objects*)

```
public class Course {
    // 1. Instance fields (must be non-static)
    private String code;
    private String name;

    // 2. Constructor (Java invokes the constructor automatically on a newly created object)
    public Course(String code, String name) {
        this.code = code;          // this.code is an instance field of the current object.
        this.name = name;          // code is a local variable in the constructor.
    }

    // 3. Instance methods (must be non-static; getters, setters, toString)
    public String getCode() {
        return code;
    }

    public String getName() {
        return name;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String toString() {
        return code + ": " + name;
    }
}
```

Now the code that is external to the ***Course*** class declaration cannot directly retrieve or change the values of the instance fields of a Course object. Instead, the code below uses a setter (*setName*) to change a field value and a getter (*getName*) to retrieve a field value.

```
Course firstCourse = new Course("swd1tf001", "Introduction to SWE");
Course currentCourse = new Course("swd4tf014", "Programming 1");

// Change the name of a course
firstCourse.setName("Introduction to software engineering");

// This time, print course names only
System.out.println(firstCourse.getName());
System.out.println(currentCourse.getName());
```

The code above prints the following:

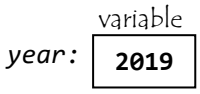
```
Introduction to software engineering
Programming 1
```

[22] In ***encapsulation***, the data held by an object (in its instance fields) is hidden from other objects of other classes, and can be accessed only through the methods of the object itself. The public methods provide an *interface* to the object.

5 Reference types, object references, and the reference copy

Java handles values of the ***primitive types*** (int, double, char, boolean etc.) directly. When you assign a value to a variable of a primitive type, the value is saved directly to the variable.

```
int year = 2019;
```

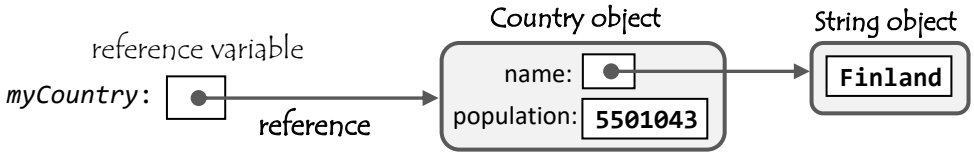


Classes and array types are known as ***reference types*** because Java handles objects and arrays by reference only. That is, ***an object itself is never stored in a variable***. When you assign an object or array to a variable, you are actually setting the variable to hold a ***reference***[23] to that object or array. If a variable contains a reference to an object, we can call the variable ***reference variable***.

Suppose that the *Country* class has instance fields called *name* (String) and *population* (int), a parametric constructor, and getters and setters for the instance fields.

```
Country myCountry = new Country("Finland", 5501043);
```

Based on the code above, Java creates the following structure:



An object exists as long as there exists a reference to the object. You cannot destroy an object explicitly. Instead, the system performs ***automatic garbage collection***. That is, the system automatically erases any unreferenced objects (called *orphans*) after some delay.

The special value `null` means an *absence of a reference* ("no reference"). `Null` can be assigned to any reference variable and it is the default value for all reference types.

```
Country myCountry;                          // The initial value of myCountry is null
System.out.println(myCountry.getName());    // ERROR: "Null Pointer Exception"

myCountry = new Country("Finland", 5542141);
System.out.println(myCountry.getName());    // Prints "Finland"

myCountry = null;
System.out.println(myCountry.getName());    // ERROR: "Null Pointer Exception"
```

NB! Only a ***reference*** to an object is copied when you copy the content of one *reference variable* to another *reference variable*. This is called ***reference copy***. An assignment operation itself does ***never*** create a new object. A new object is created with the ***new*** operator only[24]. The example below demonstrates how the ***reference copy*** works.

```
Country bigCountry = new Country("India", 1388232693);
Country smallCountry = new Country("Tuvalu", 9863);

System.out.println(bigCountry.getPopulation());     // Prints 1388232693

smallCountry = bigCountry;                          // This is a reference copy!
smallCountry.setPopulation(9864);

System.out.println(smallCountry.getPopulation());   // Prints 9864
System.out.println(bigCountry.getPopulation());     // Prints 9864     WHY?
```

Question: Can you figure out why the last print operation prints 9864 as the population? Can you draw the variables and objects on paper based on the code above?

[23] Behind the scenes, ***a reference is the memory address at which the object or array is stored***.
[24] There are two exceptions to the rule. The system automatically creates new array objects from array initializers (e.g. {1, 2, 3}) and provides a reference to a String object per each String literal (e.g. "Hello!").

6 Arrays of object references

Arrays can store references to objects in addition to primitive values[25]. When you assign an object to an array element, you are actually setting the array element to hold a reference to that object.

The easiest way of creating an array of object references is to use an array initializer. In the example below, we create three Course objects[26] and assign their references to the array.

```
Course[] courseArray = {
    new Course("swd4tf014", "Programming 1"),
    new Course("swd4tf003", "Data Management and Databases"),
    new Course("swd8tf040", "Database Developer")};
```

Creating an array with the **new** operator does ***not*** create the objects. Instead, each element of the array is initialized to null.

Another way to create an array of object references is with the ***new*** operator. In this case, Java initializes the array elements with nulls.

Please notice that *the new operator always creates exactly one object*. That is, creating an *array of Courses* does ***not*** create Course objects themselves. The Course objects must be created separately and their references should be assigned to the array elements.

```
Course[] courseArray = new Course[3];

courseArray[0] = new Course("swd4tf014", "Programming 1");
courseArray[1] = new Course("swd4tf003", "Data Management and Databases");
courseArray[2] = new Course("swd8tf040", "Database Developer");
```

Let's print names of all courses. When we iterate the array, we can copy each object reference to a separate reference variable and then use the variable to access the object.

```
for (int i = 0; i < courseArray.length; i++) {
    Course courseObject = courseArray[i];
    System.out.println(courseObject.getName());
}
```

Alternatively, we can refer to an object by indexing the array as follows:

```
for (int i = 0; i < courseArray.length; i++) {
    System.out.println(courseArray[i].getName());
}
```

We can also use the ***enhanced for loop*** to iterate the array.

```
for (Course courseObject : courseArray) {
    System.out.println(courseObject.getName());
}
```

Each of the three loops above prints the following:

```
Programming 1
Data Management and Databases
Database Developer
```

Question: Can you figure out what the code below prints?

```
Course courseObject = courseArray[0];
courseObject.setName("Hello!");

System.out.println(courseArray[0].getName());
```

[25] Strictly speaking, instead of "arrays of objects" we have *arrays of object references* in Java.
[26] We reuse the Course class declaration from a previous example.

7 Lists of object references (using the ArrayList class)

Normally, arrays are of a fixed length. That is, when you create an array you must know in advance how many items needs to be saved in the array. In Java, an ***ArrayList*** is a dynamic data structure that can grow and shrink[27]. Typically, you use an *ArrayList* instead of an array when you do not know the number of items in advance.

First, we have to import the `java.util.ArrayList` class. Then we can create a new ArrayList as follows:

```
ArrayList<Course> courseList = new ArrayList<Course>();
```

In the code above, `<Course>` specifies that only references to Course objects are allowed in this ArrayList. Once we have created an ArrayList object, we can append elements to the end of the list with the ***add*** method:

```
courseList.add(new Course("swd4tf014", "Programming 1"));
```

In Java, lists do not allow the normal array indexing operation. The neatest way of accessing the items in a List is to use the ***enhanced for loop***. The code below prints names all courses.

```
for (Course courseObject : courseList) {
    System.out.println(courseObject.getName());
}
```

If you use the ***normal for loop*** to iterate an ArrayList, then you can get the element value at the specified index by calling the **get** method of the ArrayList object.

```
for (int i = 0; i < courseList.size(); i++) {
    System.out.println(courseList.get(i));
}
```

The comprehensive example below shows the basic use of the ArrayList.

CourseListExample.java

```
import java.util.ArrayList;

public class CourseListExample {

    public static void main(String[] args) {
        ArrayList<Course> courseList = new ArrayList<Course>();

        courseList.add(new Course("swd4tf014", "Programming 1"));
        courseList.add(new Course("swd4tf003", "Data Management and Databases"));
        courseList.add(new Course("swd8tf040", "Database Developer"));

        for (Course courseObject : courseList) {
            System.out.println(courseObject.getName());
        }
    }
}
```

The *ArrayList* class provides a lot of functionality over its contents. It provides methods to insert a new item at the specified position in the list, remove an item, empty the list, sort the list etc.

[27] An ArrayList in Java is like an 'array' in JavaScript.

8 Wrapper classes

Primitive wrapper classes have the following two main uses:

1. They provide a mechanism for wrapping primitive values into corresponding objects
2. They provide useful static methods for converting Strings into numeric values

Owing to performance considerations, values of *primitive data types* (`int`, `double`, `char` ...) are not objects in Java. This presents a challenge when we are using a primitive value, but the Java method to be called requires a reference to an object as parameter. As a workaround, Java provides a ***wrapper class*** for each primitive type[28]. A primitive wrapper class allows a primitive value to be managed as an object.

For example, the wrapper class **`Integer`** represents a simple integer value. An **`Integer`** object can store a single **`int`** value. Java can automatically convert between primitive values and corresponding wrapper objects and vice versa. This is called ***autoboxing*** / ***autounboxing***.

```
Integer integerObj = 15;              // Autoboxing; Java creates an Integer object
int x = integerObj;                   // Autounboxing; Java creates an int value

System.out.println(integerObj);       // Java calls the object's toString method
System.out.println(integerObj + 1);   // Autounboxing; Java creates an int value
```

Java does not allow us to directly save primitive values in an **`ArrayList`**. Instead, we can take advantage of the corresponding wrapper class (**`Integer`**) as below.

```
ArrayList<Integer> integerList = new ArrayList<Integer>();
int sum = 0;

integerList.add(5);                          // Autoboxing;  int → Integer
integerList.add(2);                          // Autoboxing;  int → Integer
integerList.add(3);                          // Autoboxing;  int → Integer

for (Integer integerObject : integerList) {
    sum = sum + integerObject;               // Autounboxing; Integer → int
}

System.out.println(sum);
```

Primitive wrapper classes also contain useful static methods and constants. To invoke a static method of a class, we do not need to create an object from that class. For example, Strings can be converted to numeric values as below.

```
int intValue = Integer.parseInt("123");
long longValue = Long.parseLong("999_999_999_999");
double doubleValue = Double.parseDouble("7.25");
```

Strings can be converted to wrapper objects of other types as follows:

```
Integer x = Integer.valueOf("123");
Long y = Long.valueOf("999_999_999_999");
Double z = Double.valueOf("7.25");
```

A string representation of an integer as a binary number and the opposite can be created as below. All numeric wrapper classes provide this type of operations for binary, octal and hex values.

```
String str = Integer.toBinaryString(5);          // returns "101"
int intValue = Integer.parseInt("1110", 2);      // returns 14
```

Each numeric wrapper class has also a set of constants, like `MAX_VALUE` and `MIN_VALUE` etc.

```
int largestIntValue = Integer.MAX_VALUE;
int smallestIntValue = Integer.MIN_VALUE;
```

[28] The primitive wrapper classes are Byte, Short, Integer, Long, Float, Double, Character, Boolean and Void. These are specially treated classes in Java. The autoboxing mechanism is applicable to these classes only.

9 Classes that have both instance-level and class-level features

Suppose that we want to create error messages with timestamps and keep count of the number of error messages that we have created. For this purpose, we declare a class for objects (*ErrorMessage*) and include the following in the class declaration:

- A private ***instance field*** (*message*) for holding an error message text in an object
- A public ***instance method*** (*getMessage*) for retrieving an error message from the object
- A public constructor for initializing the error message field of an object
- A private ***class field***[29] (*messageCount*) for keeping the count of ErrorMessage objects
- A public ***class method*** (*getMessageCount*)for retrieving the count of ErrorMessage objects.

The complete Java class declaration is as below.

ErrorMessage.java

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ErrorMessage {
    private static int messageCount = 0;     // Class field (static)
    private String message;                  // Instance field (non-static)

    public ErrorMessage(String messageText) {
        LocalDateTime now = LocalDateTime.now();
        String dateAndTime =
            now.format(DateTimeFormatter.ofPattern("yyyy-MM-dd hh:mm:ss"));

        this.message = dateAndTime + " " + messageText;
        messageCount++;
    }

    public static int getMessageCount() {  // Class method (static)
        return messageCount;
    }

    public String toString() {      // Instance method (non-static)
        return message;
    }
}
```

Now we can create error messages, print the messages, and print the count of messages.

```
ErrorMessage firstMessage = new ErrorMessage("Driving too fast.");
ErrorMessage secondMessage = new ErrorMessage("Engine overheated.");

System.out.println(ErrorMessage.getMessageCount() + " messages created");

System.out.println(firstMessage);
System.out.println(secondMessage);
```

The code above prints the following[30]:

```
2 messages created
9.7.2019 9:22:22 Driving too fast.
9.7.2019 9:22:23 Engine overheated.
```

[29] **A class field** is associated with the class in which it is defined, rather than with an instance of the class. A class field is a *single variable* that is shared among all objects of a class. That is, a class field is visible in all methods of the class.

NB! The code in instance methods of a class can access also the class fields and invoke the class methods of the class, but the code in a class method of a class ***cannot*** access the instance fields and invoke instance methods of the class.

[30] Of course, timestamps will be different when the code is executed later again.

10 Introduction to *interfaces* and *lambda expressions*: sorting arrays and lists

10.1 Sorting arrays of primitive types

We can very easily sort an array of values of any *primitive type* with the `sort` method of the `Arrays` class. To use the class you must import `java.util.Arrays`

```
int[] numberArray = {4, 1, 2, 3};
Arrays.sort(numberArray);
```

The *sort* method is overloaded. There is a separate version of the method for sorting an array of each primitive type. In addition, there are other versions for sorting arrays of object references.

```
public class Arrays {
    private Arrays() {      // Suppresses default constructor, ensuring non-instantiability.
    }
    public static void sort(int[] array) {  //...
    }
    public static void sort(double[] array) {  //...
    }
    // ...
}
```

When a sort algorithm manipulates a given set of values, it must be able to compare two values. In the case of values of primitive types, the comparison of values is straightforward: "if (a > b)". But, ***operators like '<' and '>' do not work with objects in Java***.

Let's reuse the ***Course*** class from a previous example and try to sort an array of Course object references.

```
Course[] courseArray = {new Course("C2", "C#"), new Course("C1", "Java")};
Arrays.sort(courseArray);
```

When we run the code, the program crashes and the system prints the following message: "java.lang.ClassCastException: Course cannot be cast to java.lang.Comparable". The sort operation fails because *Java does not know how to compare Course objects*.

But, ... Strings are objects and we can sort an array of String object references as below.

```
String[] stringArray = {"Frank", "Katherine", "Aaron"};
Arrays.sort(stringArray);
```

Strings cannot be compared with operators. Instead, we must do string comparison[31] as follows:

```
if (firstWord.compareTo(secondWord) < 0) {
    System.out.println(firstWord + ", " + secondWord);
} else {
    System.out.println(secondWord + ", " + firstWord);
}
```

The `String` class has a method called ***compareTo*** that compares this String to another String. The method returns a ***negative integer*** if this String is before the other String in the ordering, a ***positive integer*** if this String is after the other String in the ordering, and ***zero*** if the strings are equal in the ordering. That is, *a String object itself knows how to compare itself to another String object*.

How does the sort method of the Arrays class work?

To compare two objects the *Arrays.sort* method calls the ***compareTo*** method of the first object and passes a reference to the second object as a parameter. Based on the return value of the method call, the *Arrays.sort* method determines if contents of the two array elements should be swapped. If the return value is greater than zero, then the first object is "greater" than the second object and the contents of the two array elements should be swapped.

[31] or compare Strings in a case-insensitive way as follows: `firstWord.compareToIgnoreCase(secondWord)`

10.2 Sorting an array with the compareTo method (using the Comparable interface)

Let's try to solve the problem of comparing Course objects by adding a method called ***compareTo*** to the Course class. The method's return values should follow the same logic as the return values of the ***compareTo*** method of the String class. We want to order courses by course code. The code of the ***compareTo*** method of the Course class is the following:

```
public int compareTo(Course anotherCourse) {
    return code.compareTo(anotherCourse.code);
}
```

In the method, we compare the course code held in this Course object to the course code held in another Course object. But, ... when we run the program, the sort operation still fails and the program crashes with the same error message as before. This is because we haven't *promised Java that our **compareTo** method can compare Course objects.*

The standard way to define a comparison method for a class and promise Java that it can compare objects created from the class is to make the class to implement the ***Comparable*** *interface*[32] as below.

Course.java (This version implements the Comparable interface)

```
public class Course implements Comparable<Course> {
    private String code;
    private String name;

    public Course(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        return code;
    }

    public String getName() {
        return name;
    }

    public int compareTo(Course anotherCourse) {
        return code.compareTo(anotherCourse.code);
    }

    public String toString() {
        return code + ": " + name;
    }
}
```

Now the sort operation succeeds gracefully!

```
Course[] courseArray = {new Course("C2", "C#"), new Course("C1", "Java")};

Arrays.sort(courseArray);

for (Course courseObject : courseArray) {
    System.out.println(courseObject);
}
```

[32] See https://docs.oracle.com/javase/8/docs/api/java/lang/Comparable.html. The intention of the ***Comparable*** interface is to provide a common mechanism for comparing one object to another. In Java, an **interface** in is an abstract type that is *used to specify a behavior that certain classes must implement*.

- The ***Comparable*** interface requires the class to implement a ***compareTo*** method that takes a single parameter of the type mentioned within the angle brackets.
- The ***compareTo*** method should return a *negative integer*, *zero*, or a *positive integer* as this object is less than, equal to, or greater than the other object.
- The *Arrays.**sort*** method approves to sort references to such objects that are created from a class that implements the ***Comparable*** interface. This guarantees that the objects have the ***compareTo*** method.

Here is another example of a class that implements the ***Comparable*** *interface*.

Circle.java (This overly simplified version of the class implements the Comparable interface)

```
public class Circle implements Comparable<Circle> {
    private int radius;

    public Circle(int radius) {
        this.radius = radius;
    }

    public int getRadius() {
        return radius;
    }

    public int compareTo(Circle anotherCircle) {
        return radius - anotherCircle.radius;
    }
}
```

10.3 Sorting an array with a Comparator object (using the Comparator interface)

If we (1) want to have more than one option for the sorting order or (2) we do not want to add the *compareTo* method to a class declaration, then we can create a separate custom class that implements the ***Comparator*** interface and a method called ***compare*** that can compare two objects of the desired type. From this class we can create a 'comparator object' and pass it to a ***sort*** method (such as **`Arrays.sort`** or **`Collections.sort`**) to allow precise control over the sort order.

CourseNameComparator.java

```
import java.util.Comparator;

public class CourseNameComparator implements Comparator<Course>
{
   public int compare(Course a, Course b) {
       return a.getName().compareTo(b.getName());
   }
}
```

```
Course[] courseArray = {new Course("C1", "Java"), new Course("C2", "C#")};

Arrays.sort(courseArray, new CourseNameComparator() );
```

10.4 Sorting an array with a lambda expression

The ***Comparator*** interface is a functional interface. A functional interface is any interface that requires only one method to be implemented (e.g. the ***compare*** method). In a case of a functional interface, we can use a ***lambda expression*** instead of creating a class for comparator objects[33]. The lambda expression is highlighted in the example below.

```
Course[] courseArray = {new Course("C1", "Java"), new Course("C2", "C#")};

Arrays.sort(courseArray, (a, b) -> a.getName().compareTo(b.getName()) );
```

A ***lambda*** or ***lambda expression*** consists of the following:

1. A comma-separated list of formal parameters enclosed in parentheses
2. The arrow token '->'
3. A body, which consists of a single expression or a statement block.

[33] To *oversimplify*: Based on the lambda expression, Java creates an anonymous class that contains the *compare* method. Java creates the body of the compare method based on the body of the lambda expression. When we run the program, Java creates an object from the anonymous class on demand and calls its compare method when needed.

In the real life, the Java compiler does some clever optimisation and delegates the construction of object representations of lambda expressions to runtime. Usually, Java does not create an anonymous class behind the scenes.

In the example below, we sort an array in descending order (z-a) with a lambda expression.

```
String[] nameArray = {"John", "Ann", "Zeke"};

Arrays.sort(nameArray, (a, b) -> b.compareTo(a));

for (String name : nameArray) {
      System.out.println(name);
}
```

A lambda expression looks like a method declaration in which we have omitted everything else but the parameters and computations.

An example of a method that compares two strings is as below.

```
public int compare(String a, String b) {
    return b.compareTo(a);
}
```

When we write a lambda expression, we can omit the access modifier, return type and method name. The arrow token '->' is mandatory in a lambda expression.

```
(String a, String b) -> {
    return b.compareTo(a);
}
```

In addition, we can omit the parameter types, braces (if a statement block is not needed) and the return keyword (if we do not use braces). Finally, we have the following lambda expression:

```
(a, b) -> b.compareTo(a)
```

If there is a single parameter only, then we can also omit the parentheses. The lambda expression below evaluates to *true* if the parameter value is an even number. Otherwise, it evaluates to *false*.

```
x -> x % 2 == 0
```

Functional programming in Java

In Java, we can also do some ***functional programming***[34] with *streams* and *lambda expressions*. In the example below, the program prints distinct even numbers in the array in ascending order. Functional programming is a *declarative* programming paradigm. For example, in the code below, there are no loops and no conditionals.

```
int[] myArray = {4, 4, 3, 8, 1, 2, 2};

Arrays.stream(myArray)
    .filter(x -> x % 2 == 0)
    .distinct()
    .sorted()
    .forEach(x -> System.out.println(x));   // Prints 2 4 8
```

A ***stream*** represents a sequence of elements and supports different kinds of operations upon those elements. ***Intermediate operations*** (e.g. `filter()`, `distinct()`, `sorted()`) return a new stream so we can chain multiple intermediate operations. ***Terminal operations*** are either void (e.g. `forEach()`) or return a non-stream result (e.g. `toArray()`).

- `Arrays.stream()` returns a stream with the specified array as its source.
- `filter()` returns a stream consisting of the elements of this stream that pass the given test.
- `sorted()` returns a stream consisting of the elements of this stream in ascending order.
- `distinct()` returns a stream consisting of the distinct elements of this stream.
- `forEach()` performs the given action for each element of this stream.

[34] See https://en.wikipedia.org/wiki/Functional_programming. To discuss the details of functional programming is not in the scope of this presentation. The given example of functional programming in Java is included here just to give you an idea where lambda expressions can be used. However, please notice how the *declarative programming approach simplifies the code* a lot in the example. Note: Lambdas are not OOP-specific constructs..

10.5 Sorting Lists

We can sort a ***list of object references*** in the same way as we can sort an array of object references. The only difference is that instead of the *sort* method of the *Arrays* class we should use the **sort** method of the ***Collections*** class. To use this class you must import `java.util.Collections`

```
ArrayList<Course> courseList = new ArrayList<Course>();

courseList.add(new Course("C2", "C#"));
courseList.add(new Course("C1", "Java"));
courseList.add(new Course("C3", "Ada"));
```

In the example below, the Course class must implement the ***Comparable*** interface and the ***compareTo*** method of the Course class must compare courses in the desired way.

```
Collections.sort(courseList);

for (Course courseObject : courseList) {
    System.out.println(courseObject);
}
```

In the example below, we use a ***lambda expression*** to compare course names. The Course class is not required to implement the *Comparable* interface and have the *compareTo* method.

```
// Sort the list in ascending order by course name
Collections.sort(courseList, (a, b) -> a.getName().compareTo(b.getName()));

for (Course courseObject : courseList) {
    System.out.println(courseObject);
}
```

10.6 A short example of a Java interface

This example shows how we can *save refrences to different types of objects in the same array*. First, we create an interface. Then, we create two different classes which implement that interface.

```
public interface Audible {                    // Create an interface called Audible
    public void makeSound();
}

public class Cat implements Audible {         // Create classes which implement
    public void makeSound() {                 // the interface. All classes which
        System.out.println("Meow!");          // implement the Audible interface
    }                                         // must have the makeSound()
}                                             // method.

public class Motorbike implements Audible {
    public void makeSound() {
        System.out.println("Vroom!");
    }
}
```

When we use a *name of an interface* as the array type, we can fill the array with references to objects that we create from *classes, which implement the interface*. NB! When we iterate the array and access the objects, we can call *only those methods that are mentioned in the interface*.

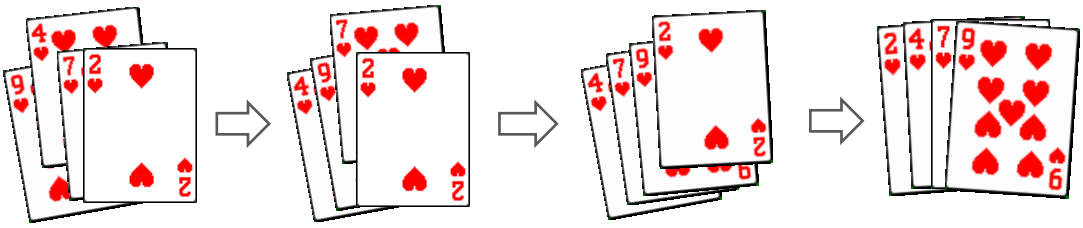
```
public class InterfaceExample {
    public static void main(String[] args) {
        Audible[] myArray = { new Cat(), new Motorbike() };

        for (int i = 0; i < myArray.length; i++) {
            myArray[i].makeSound();
        }
    }
}
```

10.7 An example of a sort algorithm

Programming languages provide efficient tools for sorting. Normally, we use these tools for sorting data in main memory. Anyhow, it is good to have some basic understanding of a sorting algorithm. There are many well-designed data sorting algorithms available for sorting data in main memory[35]. Let's look at a simple data sorting algorithm called ***insertion sort***[36].

The insertion sort algorithm works in a similar way to arranging a hand of playing cards.



InsertionSortExample.java (An example of a sort algorithm)

```
public class InsertionSortExample {
    public static void main(String[] args) {
        int[] myArray = {9, 4, 7, 2};

        sort(myArray);

        for (int i = 0; i < myArray.length; i++) {
            System.out.print(myArray[i] + " ");
        }
    }

    public static void sort(int array[]) {
        // The outer loop iterates the array once, starting from the second element.
        for (int outerIndex = 1; outerIndex < array.length; outerIndex++) {
            int currentItem = array[outerIndex];
            int innerIndex = outerIndex - 1;

            // The inner loop compares items on the left side of the 'current item', starting from the
            // last item before the current item. All the items that are greater than the 'current item'
            // are moved one step to the right. NB! The items on the left side of the current item are
            // always in ascending order.
            while (innerIndex >= 0 && array[innerIndex] > currentItem) {
                array[innerIndex + 1] = array[innerIndex];
                innerIndex--;
            }

            // Put the 'current item' to its proper location
            array[innerIndex + 1] = currentItem;
        }
    }
}
```

	[0]	[1]	[2]	[3]	array.length: **4**	
array:	9	**4**	7	2	outerIndex: **1**	currentItem: **4**
array:	4	9	**7**	2	outerIndex: **2**	currentItem: **7**
array:	4	7	9	**2**	outerIndex: **3**	currentItem: **2**
array:	2	4	7	9		

[35] See https://en.wikipedia.org/wiki/Sorting_algorithm

[36] See https://en.wikipedia.org/wiki/Insertion_sort. ***Insertion sort*** is relatively efficient for small arrays. The algorithm itself is pretty easy to understand (see the animation). The more efficient data sorting algorithms (e.g. Quick sort) use a more advanced programming technique called *recursion* that is out of scope of this presentation.

11 Generic classes: ArrayList under the hood (for advanced learners)

A data structure, such as an array or **`ArrayList`** can be understood *independently* of the element type it manipulates. A ***generic class*** (*parameterized class*) can be used to describe the concept of an `ArrayList` in *type-independent* manner.

In a declaration of a generic class, the class name is followed by a comma-separated list of ***type parameters*** in angle brackets. Within the class declaration, we can use these type parameters almost anywhere where we can use a class name. However, we cannot use a plain type parameter to create a new object or allocate memory for an array (e.g. `new T()` and `new T[5]` are illegal). When we create an object from a generic class, we write a class name in angle brackets after the name of the generic class as follows: ArrayList**<Course>** courseList = new ArrayList**<Course>**();

In the example below, the type parameter **E** after the class name represents the element type[37] of the ArrayList2. In the body of the class, we use the type parameter **E** instead of any class type whenever we should mention the type of an element of the ArrayList2.

ArrayList2.java (An oversimplified example of a generic class)

```
import java.util.Arrays;

public class ArrayList2<E> {                                    // 1.
    private Object[] array = new Object[10];                    // 2.
    private int size = 0;                                       // 3.

    public void add(E item) {                                   // 4.
        if (size == array.length) {                             // 5.
            array = Arrays.copyOf(array, array.length + 10);
        }
        array[size] = item;
        size++;
    }

    public E get(int itemIndex) {                               // 6.
        return (E)(array[itemIndex]);                           // 7.
    }

    public int size() {
        return size;
    }
}
```

1. **E** is the name of the ***type parameter*** of the class `ArrayList2`.
2. We use an array to hold object references. An array element whose type is **`Object`** can hold a reference to an object of any type. The initial number of elements in the array is 10.
3. The variable **`size`** holds the current count of items in the list. The initial count is zero.
4. We use **E** to specify the type of the method's parameter.
5. If the array is already full, we make a bigger copy of the array. We allocate 10 more elements.
6. We use **E** to specify the return type of the method.
7. We use **E** to cast the type from Object to the appropriate type.

In the code below, we use the ArrayList2 class to create a list of Strings and a list of Courses.

```
ArrayList2<String> stringList = new ArrayList2<String>();
ArrayList2<Course> courseList = new ArrayList2<Course>();

stringList.add("Hello!");
courseList.add(new Course("swd4tf014", "Programming 1"));

System.out.println(stringList.get(0));
System.out.println(courseList.get(0));
```

Please notice that the objective of the above example is just to *show the basic idea of a generic class* in Java. The code of the actual `java.util.ArrayList` class is more complex and robust.

[37] To oversimplify: A type parameter acts as a 'placeholder' for a name of an actual concrete class.
The Java naming convention for type parameters is the following: **E** for "element", **T** for "type", and **V** for "value".

List of key concepts. List of Java keywords and Java API classes used in the examples

Term	Comments and wrap-up
autoboxing	- automatic conversion between a primitive value and corresponding wrapper object
class	- *program class* | *method library class* | *class for objects*
class for objects	- the Java construct to implement a user-defined data type, providing a template to create and manipulate objects
class members	- fields, methods, constructors etc.
field	- a variable defined outside of all methods of the class
class field	- a single class-level **`static`** variable that is shared among all objects of the class
instance field	- an object-level variable, each object has a separate copy of the field
class method	- a class-level **`static`** method, requires no object to be created
instance method	- an object-level method, invoked with respect to a particular object
constructor	- *object constructor*, executed automatically when an object is created
getter	- a public accessor method that returns the value of a field (or a derived value)
setter	- a public mutator method that changes the value of a field
toString	- a public method that returns the string representation of the object
constant	- a `static final` variable, it cannot be changed once initialized
constructor overloading	- a class can have several constructors with different parameter lists
encapsulation	- protecting the state of objects with *private fields* and *public methods*
enhanced for loop	- e.g. for (Course course : courseList) { System.out.println(course); }
garbage collection	- the automatic background process of deleting unreferenced objects (orphans)
interface	- specifies a set of methods a class must have if it implements the interface
lambda expression	- In Java, *lambdas* can be passed and returned, as they were objects. They can simplify code by replacing single-method objects. E.g. `(x, y) -> x - y`
null	- null represents a reference to no object.
object	- an instance of a Java class, can have *behaviors* (methods) and *attributes* (fields)
object-oriented programming (OOP)	
primitive data type	- a non-class data type; int, double, char, boolean etc.
reference	- *object reference*, acts as a link to an object
reference copy	- only a reference (a link) is copied, no object is copied
reference type	- classes, arrays, interfaces; value of reference type is a reference to an object
reference variable	- *object reference variable*; a variable that can hold a reference to an object
visibility modifier	- *access modifier*; public, private etc.
wrapper class	- A *primitive wrapper class* allows a primitive value to be managed as an object.

Selected Java keywords

```
class        implements      interface
private      public          static
new          null
```

Special Java API classes and their methods used in the examples

Class	Method	Class	Method

`ArrayList`	`add`
	`get`
	`size`
		`Integer`	`parseInt`
			`toBinaryString`
			`valueOf`

`Arrays` `Collections`	`sort` `sort`	`Long`	`parseLong` `valueof`

`LocalDateTime`
`LocalDateTimeFormatter`
	`now`
	`format`
	`ofPattern`
		`Double`	`parseDouble`
			`valueOf`

Interface	Method
`Comparable`	`compareTo`
`Comparator`	`compare`

Please make sure that you understand each concept and keyword mentioned above.